Name: J.chandra hasini

Roll_No:2520030086

Sub: practical 3 & 4

3.

```
hasini@DESKTOP-UTCBQ51: ~
hasini@DESKTOP-UTCBQ51:~$ cd ~
mkdir -p practical3
cd practical3
hasini@DESKTOP-UTCBQ51:~/practical3$ nano practical3.c
```

```
hasini@DESKTOP-UTCBQ51: ~
 GNU nano 8.7.1                    practical3.c *
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main() {
    pid_t pid;

    printf("Parent process started\n");
    printf("Parent PID  : %d\n", getpid());
    printf("Parent PPID : %d\n\n", getppid());

    pid = fork();

    if (pid < 0) {
        perror("fork failed");
        return 1;
    }

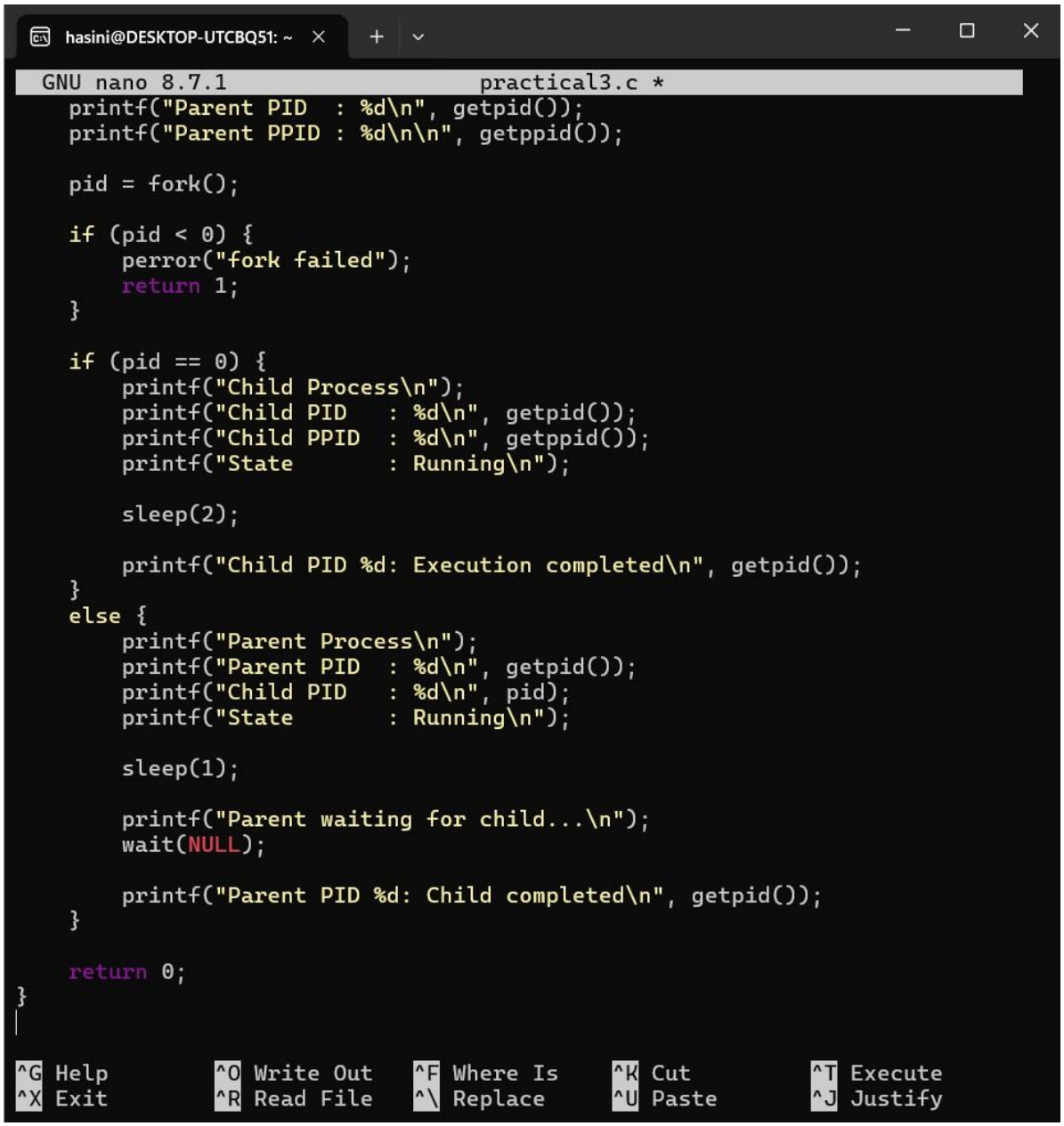
    if (pid == 0) {
        printf("Child Process\n");
        printf("Child PID   : %d\n", getpid());
        printf("Child PPID  : %d\n", getppid());
        printf("State       : Running\n");

        sleep(2);

        printf("Child PID %d: Execution completed\n", getpid());
    }
    else {
        printf("Parent Process\n");
        printf("Parent PID  : %d\n", getpid());
        printf("Child PID   : %d\n", pid);
        printf("State       : Running\n");

        sleep(1);

^G Help        ^O Write Out   ^F Where Is    ^K Cut         ^T Execute
^X Exit        ^R Read File   ^\ Replace     ^U Paste       ^J Justify
```


hasini@DESKTOP-UTCBQ51: ~
GNU nano 8.7.1 practical3.c *
printf("Parent PID : %d\n", getpid());
printf("Parent PPID : %d\n\n", getppid());
pid = fork();
if (pid < 0) {
perror("fork failed");
return 1;
}
if (pid == 0) {
printf("Child Process\n");
printf("Child PID : %d\n", getpid());
printf("Child PPID : %d\n", getppid());
printf("State : Running\n");
sleep(2);
printf("Child PID %d: Execution completed\n", getpid());
}
else {
printf("Parent Process\n");
printf("Parent PID : %d\n", getpid());
printf("Child PID : %d\n", pid);
printf("State : Running\n");
sleep(1);
printf("Parent waiting for child...\n");
wait(NULL);
printf("Parent PID %d: Child completed\n", getpid());
}
return 0;
}
^G Help ^O Write Out ^F Where Is ^K Cut ^T Execute
^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify

```
hasini@DESKTOP-UTCBQ51:~/practical3$ gcc practical3.c -o practical3
hasini@DESKTOP-UTCBQ51:~/practical3$ ./practical3
Parent process started
Parent PID  : 593
Parent PPID : 360

Parent Process
Parent PID  : 593
Child PID   : 594
State       : Running
Child Process
Child PID   : 594
Child PPID  : 593
State       : Running
Parent waiting for child...
Child PID 594: Execution completed
Parent PID 593: Child completed
hasini@DESKTOP-UTCBQ51:~/practical3$
```

4.

```
hasini@DESKTOP-UTCBQ51: ~
hasini@DESKTOP-UTCBQ51:~$ cd ~
mkdir -p practical4
cd practical4
nano practical4.c
```

```
hasini@DESKTOP-UTCBQ51: ~
  GNU nano 8.7.1                    practical4.c *
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <stdlib.h>

int main() {
    pid_t child1, child2;

    printf("Parent Process Started\n");
    printf("Parent PID: %d\n\n", getpid());

    // Create first child
    child1 = fork();

    if (child1 < 0) {
        perror("fork failed");
        return 1;
    }

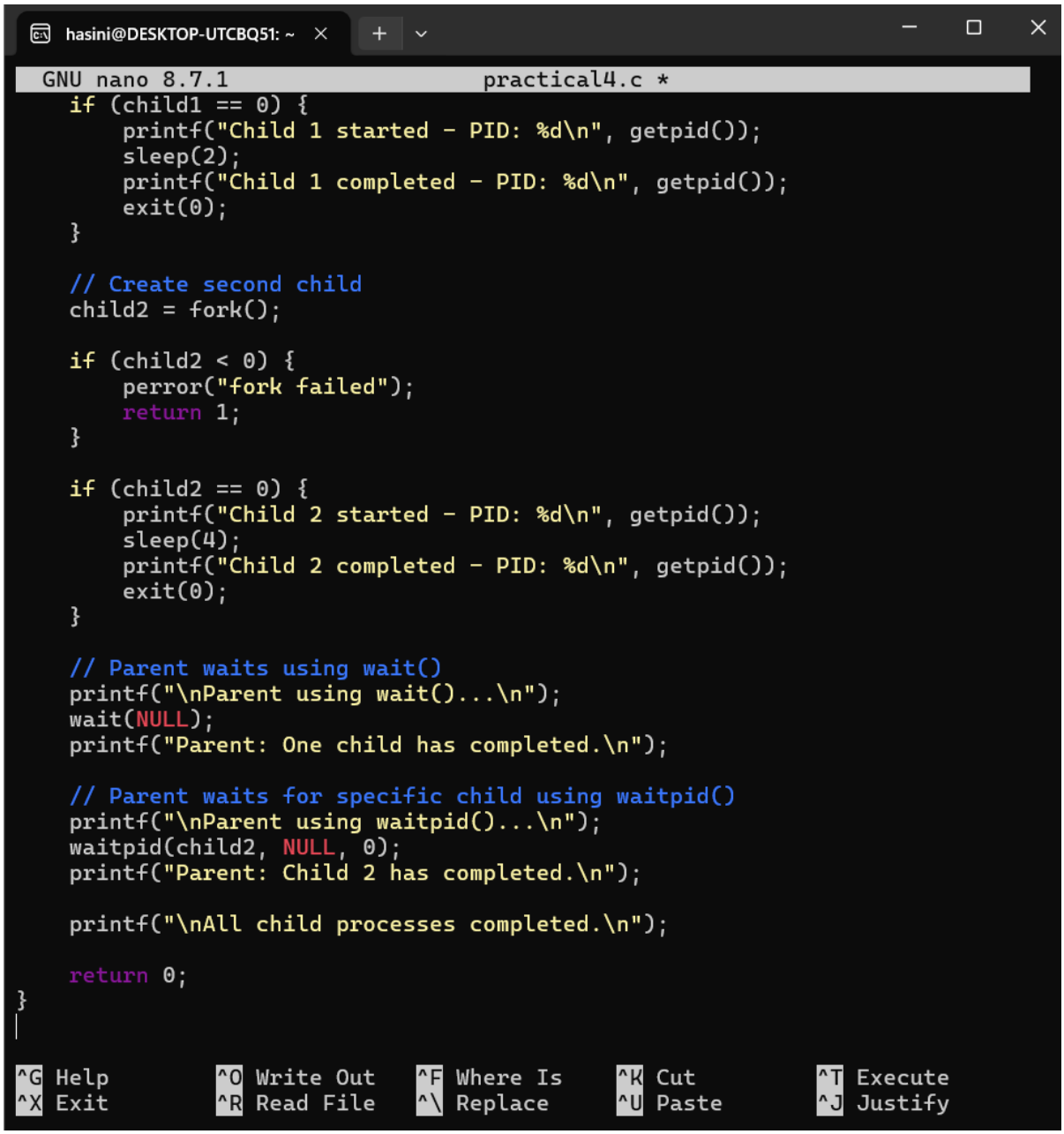
    if (child1 == 0) {
        printf("Child 1 started - PID: %d\n", getpid());
        sleep(2);
        printf("Child 1 completed - PID: %d\n", getpid());
        exit(0);
    }

    // Create second child
    child2 = fork();

    if (child2 < 0) {
        perror("fork failed");
        return 1;
    }

    if (child2 == 0) {
        printf("Child 2 started - PID: %d\n", getpid());
        sleep(4);

^G Help        ^O Write Out   ^F Where Is    ^K Cut         ^T Execute
^X Exit        ^R Read File   ^\ Replace     ^U Paste       ^J Justify
```


hasini@DESKTOP-UTCBQ51: ~
GNU nano 8.7.1 practical4.c *
if (child1 == 0) {
printf("Child 1 started - PID: %d\n", getpid());
sleep(2);
printf("Child 1 completed - PID: %d\n", getpid());
exit(0);
}
// Create second child
child2 = fork();
if (child2 < 0) {
perror("fork failed");
return 1;
}
if (child2 == 0) {
printf("Child 2 started - PID: %d\n", getpid());
sleep(4);
printf("Child 2 completed - PID: %d\n", getpid());
exit(0);
}
// Parent waits using wait()
printf("\nParent using wait()...\n");
wait(NULL);
printf("Parent: One child has completed.\n");
// Parent waits for specific child using waitpid()
printf("\nParent using waitpid()...\n");
waitpid(child2, NULL, 0);
printf("Parent: Child 2 has completed.\n");
printf("\nAll child processes completed.\n");
return 0;
}
^G Help ^O Write Out ^F Where Is ^K Cut ^T Execute
^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify

```
hasini@DESKTOP-UTCBQ51:~/practical4$ gcc practical4.c -o practical4
hasini@DESKTOP-UTCBQ51:~/practical4$ ./practical4
Parent Process Started
Parent PID: 1058

Child 1 started - PID: 1059

Parent using wait()...
Child 2 started - PID: 1060
Child 1 completed - PID: 1059
Parent: One child has completed.

Parent using waitpid()...
Child 2 completed - PID: 1060
Parent: Child 2 has completed.

All child processes completed.
hasini@DESKTOP-UTCBQ51:~/practical4$
```